

**Department of Artificial Intelligence and Data Science****Vision of the Department**

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering:

- *Dynamic learning environment that encourages problem-solving using emerging technology*
- *National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry-institute collaboration*
- *Holistic, ethical and sustainable development for lifelong success.*

Session 2025-2026

Vision: Dream of where you want.	Mission: Means to achieve Vision
---	---

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	Preparation	P: Preparation	Pep-CL abbreviation pronounce as Pep-si-IL easy to recall
PEO2	Core Competence	E: Environment (Learning Environment)	
PEO3	Breadth	P: Professionalism	
PEO4	Professionalism	C: Core Competence	
PEO5	Learning Environment	L: Breadth (Learning in diverse areas)	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:

Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

“I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life.” to contribute to the development of cutting-edge technologies and Research.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Name and Signature of Student and Date

Krishna Mishra (15/09/2026)



Department of Artificial Intelligence and Data Science

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering:

- *Dynamic learning environment that encourages problem-solving using emerging technology*
- *National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry-institute collaboration*
- *Holistic, ethical and sustainable development for lifelong success.*

Session	2025-26 (ODD)	Course Name	LAB:Neural Networks and Deep Learning
Semester	7th	Course Code	23ADS1704
Roll No	41	Name of Student	Krishna Mishra

Practical Number	06
Course Outcome	CO1: Understand the fundamental concepts of deep learning and neural network architectures CO2: Apply different techniques to develop and optimize deep learning models. CO3: Analyse the performance of deep learning models using appropriate evaluation techniques. CO4: Design and implement deep learning solutions for real-world problems using modern tools and frameworks.
Aim	Implement an LSTM-based Autoencoder using Google Colab.
Theory	An LSTM Autoencoder is a neural network that learns to encode a sequence into a compact representation and then decode it to reconstruct the original sequence. LSTM is suitable for time-series data because it can learn dependencies between observations over time. The encoder compresses the input sequence into a fixed-size representation, while the decoder reconstructs the original sequence. The model is trained using Mean Squared Error (MSE) between the input and reconstructed output. After training, reconstruction error is calculated for each sequence. If the error is greater than a selected threshold, the sequence is classified as an anomaly. This technique is useful for IoT monitoring, machine fault detection, and sensor anomaly detection.



Department of Artificial Intelligence and Data Science

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering:

- Dynamic learning environment that encourages problem-solving using emerging technology
- National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry-institute collaboration
- Holistic, ethical and sustainable development for lifelong success.

Procedure and Execution

Algorithm:

- Import NumPy, Pandas, Matplotlib, Scikit-learn and TensorFlow/Keras libraries.
- Generate synthetic IoT temperature data.
- Add abnormal temperature values to create anomalies.
- Store the data in a DataFrame.
- Normalize the temperature values using **MinMaxScaler**.
- Divide the time-series data into sequences of **30 time steps**.
- Create the LSTM Autoencoder.
- Use an LSTM layer as the encoder.
- Use **Repeat Vector** to repeat the encoded representation.
- Use another LSTM layer as the decoder.
- Reconstruct the input using **Time Distributed(Dense())**.
- Compile the model using Adam optimizer and MSE loss.
- Train the model using the input sequences as both input and target.
- Calculate reconstruction error for every sequence.
- Calculate an anomaly threshold using mean + 3 × standard deviation.
- Classify sequences with error greater than the threshold as anomalies.
- Visualize reconstruction error and the anomaly threshold.

Code:

```
import pandas as pd

# Load the dataset
df = pd.read_csv('/content/powerconsumption.csv')

# Display the first 5 rows of the DataFrame
print("First 5 rows of the dataset:")
display(df.head())

# Display information about the DataFrame, including data types and non-null values
print("\nDataFrame Info:")
df.info()

# Display descriptive statistics of the numerical columns
print("\nDescriptive Statistics:")
display(df.describe())

import matplotlib.pyplot as plt
```



Department of Artificial Intelligence and Data Science

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering:

- Dynamic learning environment that encourages problem-solving using emerging technology
- National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry-institute collaboration
- Holistic, ethical and sustainable development for lifelong success.

	<pre>import seaborn as sns # Convert 'Datetime' column to datetime objects and set as index df['Datetime'] = pd.to_datetime(df['Datetime']) df = df.set_index('Datetime') # Check for missing values print("\nMissing values per column:") print(df.isnull().sum()) # Visualize Power Consumption for each zone plt.figure(figsize=(18, 10)) plt.subplot(3, 1, 1) plt.plot(df['PowerConsumption_Zone1'], label='Zone 1') plt.title('Power Consumption - Zone 1') plt.xlabel('Date') plt.ylabel('Power Consumption') plt.legend() plt.subplot(3, 1, 2) plt.plot(df['PowerConsumption_Zone2'], label='Zone 2', color='orange') plt.title('Power Consumption - Zone 2') plt.xlabel('Date') plt.ylabel('Power Consumption') plt.legend() plt.subplot(3, 1, 3) plt.plot(df['PowerConsumption_Zone3'], label='Zone 3', color='green') plt.title('Power Consumption - Zone 3') plt.xlabel('Date') plt.ylabel('Power Consumption') plt.legend() plt.tight_layout() plt.show() from sklearn.preprocessing import MinMaxScaler import numpy as np # Select features for the LSTM autoencoder (all power consumption zones) features = ['PowerConsumption_Zone1', 'PowerConsumption_Zone2', 'PowerConsumption_Zone3'] data = df[features].values # Scale the data scaler = MinMaxScaler() data_scaled = scaler.fit_transform(data) # Define sequence length</pre>
--	---

**Department of Artificial Intelligence and Data Science****Vision of the Department**

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering:

- *Dynamic learning environment that encourages problem-solving using emerging technology*
- *National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry-institute collaboration*
- *Holistic, ethical and sustainable development for lifelong success.*

	<pre>TIME_STEPS = 30 # For example, use the last 30 time steps to predict the next # Function to create sequences def create_sequences(data, time_steps): X = [] for i in range(len(data) - time_steps): X.append(data[i:(i + time_steps)]) return np.array(X) # Create sequences for the scaled data X_sequences = create_sequences(data_scaled, TIME_STEPS) print(f"Original data shape: {data.shape}") print(f"Scaled data shape: {data_scaled.shape}") print(f"Shape of sequences for LSTM: {X_sequences.shape}") print(f"First sequence example:\n{X_sequences[0]}") from tensorflow.keras.models import Sequential from tensorflow.keras.layers import LSTM, RepeatVector, TimeDistributed, Dense # Define the input shape (TIME_STEPS, number of features) INPUT_SHAPE = (X_sequences.shape[1], X_sequences.shape[2]) # Build the LSTM Autoencoder model model = Sequential([# Encoder LSTM(units=128, activation='relu', input_shape=INPUT_SHAPE, return_sequences=False), RepeatVector(n=INPUT_SHAPE[0]), # Repeat the last output of the encoder # Decoder LSTM(units=128, activation='relu', return_sequences=True), TimeDistributed(Dense(units=INPUT_SHAPE[1])) # Output a Dense layer for each time step]) # Compile the model model.compile(optimizer='adam', loss='mean_squared_error') # Display model summary print("LSTM Autoencoder Model Summary:") model.summary() from tensorflow.keras.callbacks import EarlyStopping # Define callbacks for early stopping to prevent overfitting early_stopping = EarlyStopping(monitor='val_loss', patience=5, mode='min', restore_best_weights=True)</pre>
--	--



Department of Artificial Intelligence and Data Science

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering:

- Dynamic learning environment that encourages problem-solving using emerging technology
- National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry-institute collaboration
- Holistic, ethical and sustainable development for lifelong success.

```
# Train the model
history = model.fit(
    X_sequences, X_sequences, # Autoencoder: input and output are the same
    epochs=20, # You can adjust this
    batch_size=128, # You can adjust this
    validation_split=0.2, # Use 20% of data for validation
    callbacks=[early_stopping], # Add early stopping callback
    shuffle=False # Maintain sequence order
)

# Plot training and validation loss
plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss During Training')
plt.xlabel('Epoch')
plt.ylabel('Loss (Mean Squared Error)')
plt.legend()
plt.grid(True)
plt.show()

from sklearn.metrics import mean_absolute_error

# Get predictions (reconstructions) from the trained model
X_pred = model.predict(X_sequences)

# Calculate reconstruction errors for each sequence
# We will calculate MAE between original and reconstructed sequences
reconstruction_errors = np.array([mean_absolute_error(X_sequences[i],
X_pred[i]) for i in range(len(X_sequences))])

# Extend the reconstruction errors to match the original dataframe's length for
plotting
# The first TIME_STEPS elements don't have a full sequence for error
calculation,
# so we pad them with zeros or NaNs. Here, we'll use a simplified approach for
visualization.
# The errors correspond to the *end* of each sequence.
full_reconstruction_errors = np.full(data_scaled.shape[0], np.nan)
full_reconstruction_errors[TIME_STEPS:] = reconstruction_errors

# Plot reconstruction errors
plt.figure(figsize=(15, 7))
plt.plot(df.index, full_reconstruction_errors, label='Reconstruction Error')
plt.title('Reconstruction Error Over Time')
plt.xlabel('Time')
plt.ylabel('Reconstruction Error (MAE)')
plt.legend()
plt.grid(True)
plt.show()
```



Department of Artificial Intelligence and Data Science

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering:

- Dynamic learning environment that encourages problem-solving using emerging technology
- National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry-institute collaboration
- Holistic, ethical and sustainable development for lifelong success.

```
import numpy as np

# Get predictions (reconstructions) from the trained model
# X_pred = model.predict(X_sequences) # This line should be before this cell,
# as it was in the original flow

# Calculate reconstruction errors for each sequence
# We will calculate MAE between original and reconstructed sequences
# reconstruction_errors = np.array([mean_absolute_error(X_sequences[i],
# X_pred[i]) for i in range(len(X_sequences))]) # This line should be before this
# cell

# Calculate statistics of reconstruction errors
mean_error = np.mean(reconstruction_errors)
std_error = np.std(reconstruction_errors)

# Determine a threshold for anomalies
# A common approach is to use a multiple of the standard deviation
threshold = mean_error + 2 * std_error # 2 standard deviations above the mean

print(f"\n--- Anomaly Detection Metrics ---")
print(f"Mean Reconstruction Error (MAE): {mean_error:.4f}")
print(f"Standard Deviation of Reconstruction Error: {std_error:.4f}")
print(f"Calculated Anomaly Threshold (Mean + 2*Std): {threshold:.4f}")

# Identify anomalies
anomalies = np.where(reconstruction_errors > threshold)[0]

# Map anomalies back to original DataFrame indices
# Note: anomalies are identified on sequences, so their timestamp corresponds
# to the end of the sequence
anomaly_dates = df.index[TIME_STEPS + anomalies]

print(f"Number of detected anomalies: {len(anomalies)}")
print("Dates of first 10 detected anomalies (if any):")
if len(anomaly_dates) > 0:
    for date in anomaly_dates[:10]: # Print first 10 anomalies if many
        print(date)
    if len(anomaly_dates) > 10:
        print("...")
else:
    print("No anomalies detected above the threshold.")

import matplotlib.dates as mdates

# Create a DataFrame for plotting anomalies
anomaly_df = pd.DataFrame(df.iloc[TIME_STEPS:].index)
anomaly_df['reconstruction_error'] = reconstruction_errors
anomaly_df['anomaly'] = anomaly_df['reconstruction_error'] > threshold
```



Department of Artificial Intelligence and Data Science

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering:

- Dynamic learning environment that encourages problem-solving using emerging technology
- National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry-institute collaboration
- Holistic, ethical and sustainable development for lifelong success.

```
# Merge anomaly information back to the original DataFrame
plot_df = df.copy()
plot_df = plot_df.iloc[TIME_STEPS:] # Align data for plotting errors
plot_df['reconstruction_error'] = reconstruction_errors
plot_df['anomaly'] = plot_df['reconstruction_error'] > threshold

# Visualize anomalies for each power consumption zone
plt.figure(figsize=(20, 15))

for i, feature in enumerate(features):
    plt.subplot(len(features), 1, i + 1)
    plt.plot(plot_df.index, plot_df[feature], label=f'{feature} (Normal)')

# Plot anomalies
anomalous_data = plot_df[plot_df['anomaly']]
if not anomalous_data.empty:
    plt.scatter(
        anomalous_data.index,
        anomalous_data[feature],
        color='red',
        label='Anomaly',
        s=20,
        marker='o'
    )

plt.title(f'Power Consumption - {feature} with Anomalies')
plt.xlabel('Date')
plt.ylabel('Power Consumption')
plt.legend()
plt.grid(True)

# Format x-axis for better date display
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m-%d'))
plt.gca().xaxis.set_major_locator(mdates.AutoDateLocator())
plt.gcf().autofmt_xdate() # Auto-format date labels

plt.tight_layout()
plt.show()
```

Output:



Department of Artificial Intelligence and Data Science

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering:

- Dynamic learning environment that encourages problem-solving using emerging technology
- National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry-institute collaboration
- Holistic, ethical and sustainable development for lifelong success.

	
Output Analysis	• The LSTM-based Autoencoder was successfully implemented in Google Colab. It learned the normal temperature patterns and detected abnormal sequences using reconstruction error. The obtained reconstruction MSE was approximately 0.00347, with an anomaly threshold of approximately 0.03039, resulting in 39 detected anomalous sequences in the provided run.
Link of student Github profile where lab assignment has been uploaded	https://github.com/krish-na45/Neural-Network-Deep-Learning-Practicals
Conclusion	The LSTM Autoencoder successfully learned the temporal patterns of IoT temperature data and reconstructed normal sequences with low error. Reconstruction error was used as the evaluation measure instead of classification accuracy because the Autoencoder performs unsupervised anomaly detection. Sequences having reconstruction error greater than the calculated threshold were identified as anomalies. Thus, LSTM Autoencoders can be effectively used for time-series anomaly detection in applications such as IoT sensors, machine monitoring, and predictive maintenance.



Department of Artificial Intelligence and Data Science

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering:

- *Dynamic learning environment that encourages problem-solving using emerging technology*
- *National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry-institute collaboration*
- *Holistic, ethical and sustainable development for lifelong success.*

Plag Report
(Similarity index < 12%)



Page 2 of 5 - Integrity Overview

Submission ID: 27005:130402451

11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

▸ Bibliography

Exclusions

▸ 6 Excluded Matches

Match Groups

- 2 Not Cited or Quoted 11%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 11% Internet sources
- 11% Publications
- 11% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Date

15/09/2026